COMP 2011 Midterm Exam - Spring 2023 - HKUST

Date: March 31, 2023 (Friday)
Time Allowed: 2 hours, 7:30-9:30pm
Instructions:

1. This is a closed-book, closed-notes examination.
2. There are **9** questions on **24** pages (including this cover page, honor code and 3 blank pages at the end).
3. Write your answers in the space provided in black/blue ink. *NO pencil please, otherwise you are not allowed to appeal for any grading disagreements.*
4. All programming codes in your answers must be written in the ANSI C++ version as taught in the class.
5. For programming questions, you are **NOT** allowed to define additional helper functions or structures, nor global variables unless otherwise stated. You **cannot** use any library functions not mentioned, nor the `auto` keyword in defining identifiers in the questions.

Student Name	
Student ID	
Email Address	
Lecture & Lab Section	

Problem	**Score**
1	/ 8
2	/ 10
3	/ 6
4	/ 9
5	/ 10
6	/ 8
7	/ 10
8	/ 18
9	/ 21
Total	/ 100

The HKUST Academic Honor Code

Honesty and integrity are central to the academic work of HKUST. Students of the University must observe and uphold the highest standards of academic integrity and honesty in all the work they do throughout their program of study.

As members of the University community, students have the responsibility to help maintain the academic reputation of HKUST in its academic endeavors.

Sanctions will be imposed if students are found to have violated the regulations governing academic integrity and honesty.

Declaration of Academic Integrity

I confirm that I have answered the questions using only materials specifically approved for use in this examination, that all the answers are my own work, and that I have not received any assistance during the examination.

Signature: _____

Problem 1 [8 points] Operator Precedence and Associativity

Operator	Description	Associativity
&&	logical AND	Left-to-Right
== !=	equal, not equal	Left-to-Right
+ −	add, subtract	Right-to-Left
* /	multiply, divide	Right-to-Left

Table 1: Operators in decreasing precedence.

This question only involves the operators shown in Table 1. Assume that the precedence and associativity of these operators are changed as shown in the table. The top row has the highest precedence and each next row has lower precedence. Note that the table is **different** from standard `C++`. Use parentheses '(' and ')' to show the order of computation of the following expressions. Identifiers, a, b, c, d, e, f in the expressions are assumed well-defined before the expressions so that there are no syntax errors nor semantic errors. For example, in the following parenthesized expression where `@` and `#` are binary operators,

```
(a @ (b # c)) @ d
```

the order of computation is as follows: the sub-expression `b#c` is computed first to give a result, say, u, then `a@u` is computed to give another result, say, v. Finally, `v@d` is computed to get the final value of the whole expression.

(a) `a + b - c`

Answer: _____

(b) `a == b && c != d`

Answer: _____

(c) `a + b == c - d`

Answer: _____

(d) `a + b * d / e - f`

Answer: _____

Problem 2 [10 points] Reference Variables

In each part below, write down (1) the output if the program can be run successfully; **or**, (2) "COMPILATION ERROR" **and** the line number of the code that causes the error, and give a brief explanation if the program cannot be compiled; **or**, (3) "RUNTIME ERROR" **and** the line number of the code that may cause the error, and give a brief explanation if the program can be compiled but may fail when it is run.

(a)

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int x = 4;
7      for (int j = x; j < 6; ++j)
8          cout << j << ' ';
9      cout << endl << x << endl;
10     return 0;
11 }
```

Answer: _____

(b)

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int x = 4;
7      for (int& j = x; j < 6; ++j)
8          cout << j << ' ';
9      cout << endl << x << endl;
10     return 0;
11 }
```

Answer: _____

(c)

```
#include <iostream>
using namespace std;

int main()
{
    int x = 4;
    for (const int& j = x; j < 6; ++j)
        cout << j << ' ';
    cout << endl << x << endl;
    return 0;
}
```

Answer: _____

(d)

```
#include <iostream>
using namespace std;

int main()
{
    const int x = 4;
    for (int& j = x; j < 6; ++j)
        cout << j << ' ';
    cout << endl << x << endl;
    return 0;
}
```

Answer: _____

Problem 3 [6 points] Parameters Passing

```
1  #include <iostream>
2  using namespace std;
3
4  int f(...) { return x += 10; } // Incomplete
5
6  int main()
7  {
8      int a = 5;
9      cout << f(a) << ' ';
10     cout << a << endl;
11     return 0;
12 }
```

In the program above, the function **f** in line#4 is incomplete as the formal parameter **x** has not been declared yet (but is denoted as "$\cdots$"). In each part below, line#4 is replaced in a different way. Write down (1) the output if the resulting program can be run successfully; **or**, (2) "COMPILATION ERROR" and give a brief explanation if the resulting program cannot be compiled; **or**, (3) "RUNTIME ERROR" and give a brief explanation if the resulting program can be compiled but may fail when it is run.

(a) `int f(int x) { return x += 10; }`

Answer: _____

(b) `int f(int& x) { return x += 10; }`

Answer: _____

(c) `int f(const int x) { return x += 10; }`

Answer: _____

Problem 4 [9 points] Scope

The following code contains 3 errors that stop it from compiling successfully.

```
1  #include <iostream>
2  using namespace std;
3
4  int a,b;
5
6  void foo(double a, double b){
7      if (a > b) bar(a,b);
8      cout << a << " ";
9  }
10
11 void bar(double &a, double &b){
12     double t = b;
13     b = a; a = t;
14 }
15
16 int main(){
17     int a = 4, b = -3; foo(a,b);
18     double a = 2.0, b = 1.1; foo(a,b);
19
20     for (int counter=0, a = 0; a < 10; a++){
21         if (a%2 == 0) counter++;
22     }
23     cout << counter;
24
25     return 0;
26 }
```

Identify each error by writing down the line number where it occurs, and briefly explain why it is an error. Then explain how to fix it so that the output is `-3 1.1 5`

Answer:

Line#	Error Description	Fix

Problem 5 [10 points] Debugging

The following program contains 5 errors. The errors may be syntax errors (i.e., compilation errors) or runtime errors.

Identify each error by writing down the line number where it occurs, and explain why it is an error. In identifying the errors, please consider them independently, and assume that the other errors have been fixed and do not exist.

```
1  #include <iostream>
2  using namespace std;
3
4  enum ALPHABET { A = 0.0, B, C };
5  const int scores[] = {5, 3, 1};
6
7  char convert(char x)
8  {
9      char c = x;
10     c =  ((c >= 'A') && (c <= 'Z')) ? (c + ('a' - 'A')) : c;
11     c =  (c < 'a') ? 'a' : ((c > 'z') ? "z" : c);
12     return c;
13 }
14
15 int calculate(const char letters[], int size)
16 {
17     int score = 0;
18     for (int i=0; i < size; ++i)
19     {
20         double x = letters[i] - 'a';
21         switch (x) {
22             case A:
23                 score += scores[A];
24                 break;
25             case B:
26                 score += scores[B];
27                 break;
28             case C:
29                 score += scores[C];
30         }
31     }
32     return score;
33 }
34
```

```
35 int main()
36 {
37     char letters[5];
38     char temp;
39     int size = sizeof(letters)/sizeof(char);
40
41     int i = 0;
42     while (i < size)
43     {
44        cin >> temp;
45        temp = convert(temp);
46        letters[++i] = temp;
47     }
48
49     cout << letters;
50     cout << "==>" << calculate(letters, size) << endl;
51     return 0;
52 }
```

Answer:

Error#	Line#	Explanation
1		
2		
3		
4		
5		

Problem 6 [8 points] C String

```
bool check_string(const char s1[], const char s2[])
{
    int j = 0;
    for (int i = 0; s1[i] != '\0'; ++i)
    {
        if (s1[i] == s2[j])
        {
            ++j;
            if (s2[j] == '\0')
                return true;
       }
    }
    return false;
}
```

Given the function above, write the return value for each of the function calls below. The following example is given to you as a reference.

`check_string("good", "best")`

Answer: false

(a) `check_string("us", "hkust")`

Answer: _____

(b) `check_string("singing", "ing")`

Answer: _____

(c) `check_string("ginger", "give")`

Answer: _____

(d) `check_string("programming001", "roam1")`

Answer: _____

(e) `check_string("computer science", "pure")`

Answer: _____

Problem 7 [10 points] Recursion

Write a *recursive* function `printOddOrEven` that takes as input a positive integer `n`. If `n` is odd, it prints all positive odd numbers that are less than or equal to `n` first in decreasing order and then in increasing order. If `n` is even, it prints all positive even numbers that are less than or equal to `n` first in decreasing order and then in increasing order. The function returns `void` and you can assume the input is a positive integer.

For example for `n` = 11 it prints `11 9 7 5 3 1 1 3 5 7 9 11`. For `n` = 22 it prints `22 20 18 16 14 12 10 8 6 4 2 2 4 6 8 10 12 14 16 18 20 22`. Every number is followed by a single whitespace. You *must not* use loops of any kind such as `for`, `while`, or `do-while`.

Answer:

Problem 8 [18 points] Arrays

Below you are given an incomplete C++ program. The purpose of the program is to print and circular rotate 1D and 2D arrays.

```
#include <iostream>
#include <iomanip>
using namespace std;

const int MAX_COLS = 8;

// Print a 1D array with a given number of columns
void print_array(const int x[], unsigned int cols)
{
    for (int i = 0; i < cols; ++i)
        cout << setw(2) << x[i] << " ";
        // setw(n) will print x[i] by width of 2
    cout << endl;
}

// Print a 2D array with a given number of rows and columns
void print_array_2d(const int x[][MAX_COLS], unsigned int rows, unsigned int cols) {
    for (int i = 0; i < rows; ++i)
        print_array(x[i], cols);
}

// Function definitions of circular_roatation()
// and circular_rotation_2d() go here ...

int main()
{
    int arr1[MAX_COLS] = {1, 2, 3, 4, 5, 6};
    int num_col1 = 6;

    cout << "Initial 1d array:" << endl;
    print_array(arr1, num_col1);
    cout << endl << "Circular rotates the elements with index < "
         << num_col1 << " by " << 2 << " steps " << endl;
    circular_rotation(arr1, num_col1, 2);
    print_array(arr1, num_col1);
    cout << endl << "Circular rotates the elements with index < "
         << num_col1/2 << " by " << 1 << " steps " << endl;
    circular_rotation(arr1, num_col1/2, 1);
    print_array(arr1, num_col1);

    int arr2[][MAX_COLS] = {{1, 2, 3, 4, 5, 6, 7},
                            {8, 9, 10, 11, 12, 13, 14},
                            {15, 16, 17, 18, 19, 20, 21}};
```

```
    int num_col2 = 7;
    cout << endl << "Initial 2d array:" << endl;
    print_array_2d(arr2, 3, num_col2);
    cout << endl << "Circular rotates the elements with row index < "
         << 3 << " and column index < " << num_col2
         << " by " << 4 << " steps " << endl;
    circular_rotation_2d(arr2, 3, num_col2, 4);
    print_array_2d(arr2, 3, num_col2);
    cout << endl << "Circular rotates the elements with row index < "
         << 2 << " and column index < " << num_col2/2
         << " by " << 2 << " steps " << endl;
    circular_rotation_2d(arr2, 2, num_col2/2, 2);
    print_array_2d(arr2, 3, num_col2);

    return 0;
}
```

You are asked to complete the implementations of two functions, namely, `circular_rotation()` and `circular_rotation_2d()` such that the program can give the corresponding output:

```
Initial 1d array:
 1  2  3  4  5  6

Circular rotates the elements with index < 6 by 2 steps
 3  4  5  6  1  2

Circular rotates the elements with index < 3 by 1 steps
 4  5  3  6  1  2

Initial 2d array:
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21

Circular rotates the elements with row index < 3 and column index < 7 by 4 steps
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21  1  2  3  4

Circular rotates the elements with row index < 2 and column index < 3 by 2 steps
 7 12 13  8  9 10 11
14  5  6 15 16 17 18
19 20 21  1  2  3  4
```

Note: Your implementations MUST work with:

- ANY value assigned to the global constant `MAX_COLS`, and
- ANY numbers of rows and columns, and
- ANY element values in the arrays.

Note: You are not allowed to define or create any additional arrays.

(a) The function `circular_rotation()` will circularly shift the elements of a given 1D array within a given number of columns to the left by a given number of steps.

You can assume the number of columns is always a positive integer value smaller than or equal to `MAX_COLS` and the number of steps is always a positive integer value.

Write the complete definition, including the function header and the function body, for the function `circular_rotation()`.

Answer:

(b) The function `circular_rotation_2d()` will circularly shift the elements of a given 2D array within given numbers of columns and rows by a given number of steps.

You can assume the number of columns is always a positive integer value smaller than or equal to `MAX_COLS`, the number of rows is always a postive integer value smaller than or equal to the maximum number of rows of the array and the number of steps is always a positive integer value.

Write the complete definition, including the function header and the function body, for the function `circular_rotation_2d()`.

Answer:

Problem 9 [21 points] Character Array and Functions

This question is about printing the characters in a C string stored in a 1D character array in different ways. You may assume that the array size is always big enough to store the string and the number of characters in the C string is always greater than zero. Based on the examples, write the code of each function according to the given header. Your solution should work for C strings of any length greater than zero.

Note: You are **not** allowed to define or create any additional arrays. On the other hand, you may add helper function(s) if you find it/them useful.

(a) A function of the following header:

```
void vprint(const char s[], int length, int num_char_per_column)
```

`vprint` prints the C string `s` over `num_char_per_column` lines. Integer `length` is the number of characters in the C string `s`, and the integer `num_char_per_column` is the number of characters to be printed per column. If `length` is not divisible by `num_char_per_column`, the last (or rightmost) column will show fewer characters. Below are some examples when `s` is "1234567". Recall that `strlen(s)` returns the character length of `s`.

Call	Output
`vprint(s, strlen(s), 1);`	1234567
`vprint(s, strlen(s), 2);`	1357 246
`vprint(s, strlen(s), 5);`	16 27 3 4 5
`vprint(s, strlen(s), 7);`	1 2 3 4 5 6 7

Answer:

```
// Assumption: 0 < length; 0 < num_char_per_column <= length
void vprint(const char s[], int length, int num_char_per_column)
```

(b) A function of the following header:

```
void hprint(const char s[], int length, int num_char_per_row)
```

`hprint` also prints over one or more lines, but it restricts the number of characters to be printed on each row to `num_char_per_row` instead. If `length` is not divisible by `num_char_per_row`, the last row will show fewer characters. Below are some examples when `s` is "abcdef".

Call	Output

`hprint(s, strlen(s), 1);`	`a`
	`b`
	`c`
	`d`
	`e`
	`f`

`hprint(s, strlen(s), 2);`	`ab`
	`cd`
	`ef`

`hprint(s, strlen(s), 5);`	`abcde` `f`
`hprint(s, strlen(s), 8);`	`abcdef`

Answer:

```
// Assumption: 0 < length; 0 < num_char_per_row
void hprint(const char s[], int length, int num_char_per_row)
```

(c) A function of the following header:

```
void odd_print(const char s[], int length)
```

In `odd_print`, the length is assumed to be an odd number. It will print the given C string `s` in inverted V manner as shown in the following examples.

Call	Output
`odd_print("A", 1);`	`A`

`odd_print("12345", 5);`	`  3`
	`2 4`
	`1  5`

`odd_print("abcdefg", 7);`	`   d`
	`  c e`
	` b  f`
	`a    g`

Some remarks about the spacing:

- the 1st row prints only the middle character in the string `s`.
- the last row prints with no leading spaces; the row above it prints with one leading space; the last but 2 row prints with 2 leading spaces, etc.
- the number of spaces between the 2 printed characters in each row (other than the 1st row) is equal to the number of characters between them in the string `s`. E.g., in the last example, 'b' and 'f' are printed in the last but one row, separated by 3 spaces as there are 3 characters between them in the given C string "abcdefg".

Answer:

```
// Assumption: 0 < length; length is odd
void odd_print(const char s[], int length)
```

---------- END OF PAPER ----------

/* Rough work — You may detach this page */

/* Rough work — You may detach this page */

/* Rough work — You may detach this page */